

# Actividad final integradora: El motor de datos de envíos de RutaSur

PF1462 · Especialización en Machine Learning

Módulo 2 · MA03776 · Programación avanzada para el aprendizaje de máquina

## MOMENTO

Al cierre del módulo, después de resolver las actividades de sus cinco contenidos

## MODALIDAD

Individual, con entrega en el LMS

## CALIFICACIÓN

pauta en puntos, exigencia 60 %

## COMPETENCIA DEL MÓDULO

Utilizar técnicas avanzadas de programación en Python para desarrollar, optimizar y mantener aplicaciones de mediana complejidad, aplicando conceptos de estructuras de datos, manejo de excepciones, programación orientada a objetos, optimización de código y programación de algoritmos.

## Contexto

RutaSur (empresa ficticia) es una empresa de despacho con seis centros de distribución entre Santiago y Puerto Montt. Quiere entrenar un modelo que anticipe qué envíos llegarán atrasados, pero antes necesita un código confiable que prepare los datos. Hoy tiene un script de un practicante: se cae cuando una fila viene mal, tarda demasiado con el histórico completo y nadie se atreve a modificarlo.

Los datos de ejemplo son dos archivos CSV (los publica el tutor/a en el LMS, o los generas tú con `numpy.random` respetando estas columnas):

| Archivo                 | Filas  | Columnas                                                                                                                                                                                                                                          |
|-------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>envios.csv</code> | 50.000 | <code>id_envio</code> , <code>fecha</code> , <code>centro_origen</code> , <code>centro_destino</code> , <code>tipo_servicio</code> (estandar, express o refrigerado), <code>peso_kg</code> , <code>distancia_km</code> , <code>dias_atraso</code> |
| <code>rutas.csv</code>  | 14     | <code>origen</code> , <code>destino</code> , <code>distancia_km</code> (conexiones viales entre los seis centros)                                                                                                                                 |

Cerca del 3 % de las filas de `envios.csv` viene con errores: peso negativo o vacío, fecha mal escrita, un centro que no existe o un `id_envio` repetido.

## El desafío

Reescribe el script como un paquete pequeño y ordenado que lea, valide, enriquezca y resuma los envíos sin caerse, que se pueda extender a nuevos tipos de servicio sin tocar lo que ya funciona y que procese el histórico completo en una fracción de lo que demora la versión actual.

## Qué tienes que entregar

- 1. Estructuras de datos.** Carga los centros y las rutas en las estructuras que correspondan y justifica cada elección en una tabla breve (lista, tupla, diccionario, conjunto, pila o cola). Representa la red de rutas como un grafo con lista de adyacencia y calcula la ruta más corta entre dos centros (Dijkstra con `heapq`). Recorre la red con una cola (`collections.deque`) para listar los centros alcanzables. Arma un árbol zona → región → centro y súmale los envíos por nodo con un recorrido recursivo.
- 2. Excepciones y registro.** Crea una jerarquía propia: `ErrorEnvio` y sus subclases `RegistroInvalidoError` y `CentroInexistenteError`. Al leer cada fila, controla los errores con `try-except-else-finally`, sin capturar `Exception` a ciegas: la fila mala se aparta, se registra con `logging` en `rutasur.log` (nivel, fila y motivo) y el proceso sigue. Explica qué excepciones dejas propagar y por qué.
- 3. Clases y objetos.** Diseña la clase `Envio`, con `peso_kg` encapsulado en una propiedad que valida el valor, y una clase abstracta `Tarifa` con tres subclases (`TarifaEstandar`, `TarifaExpress` y `TarifaRefrigerada`) que redefinen `calcular_costo()`. Usa la herencia múltiple en un caso justificado (por ejemplo, un *mixin* que exporta a diccionario). Muestra que agregar un cuarto servicio no obliga a modificar las clases existentes y explica, con un ejemplo de tu código, cómo aplicaste DRY, KISS, sustitución de Liskov, abierto/cerrado y segregación de interfaces.
- 4. Análisis de algoritmos.** El script actual busca duplicados comparando cada envío con todos los demás y ubica un envío recorriendo la lista completa. Expresa la complejidad temporal y espacial de ambas funciones con notación Big O, en caso promedio y peor caso. Reemplázalas por versiones eficientes (conjunto o diccionario, y búsqueda binaria con `bisect` sobre la lista ordenada), implementa un ordenamiento recursivo (*merge sort*) y compáralo con `sorted()`. Mide antes y después con `timeit` y `cProfile` y presenta los resultados en una tabla.
- 5. Optimización del código.** Lee el archivo con un generador dentro de un *context manager* (`with`), usa comprensiones de listas o diccionarios donde aporten claridad y vectoriza con NumPy y pandas el cálculo del costo y el resumen de atraso por ruta y tipo de servicio (`groupby`). Procesa con Dask una versión de `envios.csv` replicada a 5 millones de filas y acelera con `@numba.njit` una función numérica de tu elección. Indica cuándo conviene cada herramienta y cuándo no.
- 6. Informe breve (PDF, 3 a 5 páginas).** Las decisiones de diseño, las tablas de los pasos 1 y 4, la comparación de rendimiento, las filas apartadas por tipo de error y un párrafo sobre qué quedaría listo para entrenar el modelo de atrasos.

**Entrega en la Tarea del LMS:** el notebook `rutasur.ipynb` o el paquete en `.zip` (con un `README` que diga cómo ejecutarlo), el archivo `rutasur.log` y el informe `informe-rutasur.pdf`. El código debe correr de principio a fin sin errores con los archivos de ejemplo y seguir la guía de estilo PEP 8.

## Pauta de evaluación

Cada fila se asigna completa si se cumple, a la mitad si se cumple en parte y en cero si no está.

| Componente de la competencia          | Qué se revisa                                                                                                                | Puntos |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------|--------|
| Estructuras de datos                  | Elección justificada de cada estructura; pila o cola usada donde corresponde                                                 | 3      |
|                                       | Grafo con lista de adyacencia, ruta más corta correcta y recorrido con cola; árbol con recorrido recursivo                   | 5      |
| Manejo de excepciones                 | Jerarquía propia de excepciones y bloques <code>try-except-else-finally</code> bien ubicados                                 | 4      |
|                                       | El proceso no se cae: filas malas apartadas y registradas con <code>logging</code> ; propagación explicada                   | 4      |
| Programación orientada a objetos      | <code>Envío</code> con atributos, métodos y propiedad encapsulada; <code>Tarifa</code> abstracta con herencia y polimorfismo | 5      |
|                                       | Herencia múltiple justificada y los cinco principios de diseño explicados con el propio código                               | 3      |
| Programación y análisis de algoritmos | Complejidad Big O temporal y espacial, en caso promedio y peor caso, de las funciones originales                             | 4      |
|                                       | Versiones eficientes, ordenamiento recursivo y mediciones con <code>timeit</code> y <code>cProfile</code>                    | 4      |
| Optimización de código                | Generador, <i>context manager</i> y comprensiones; cálculo vectorizado con NumPy y pandas                                    | 4      |
|                                       | Dask y Numba aplicados y comparados, con criterio sobre cuándo usarlos                                                       | 2      |
| Calidad de la entrega                 | Código que corre de principio a fin, legible (PEP 8) e informe claro                                                         | 2      |
| Total                                 |                                                                                                                              | 40     |

**Nota** (exigencia 60 %): si el puntaje  $p$  es mayor o igual que 24,  $\text{nota} = 4,0 + 3 \times (p - 24) / 16$ ; si es menor,  $\text{nota} = 1,0 + 3 \times p / 24$ . Se redondea a un decimal.